IES Velázquez
Sevilla

UNIDAD 5

Script Shell Linux



SISTEMAS INFORMÁTICOS

Técnico de Grado Superior Desarrollo de Aplicaciones Web

1. Nuestro primer Script

Script = fichero de texto ASCII puro, que pueden ser creados por cualquier editor del que dispongamos.

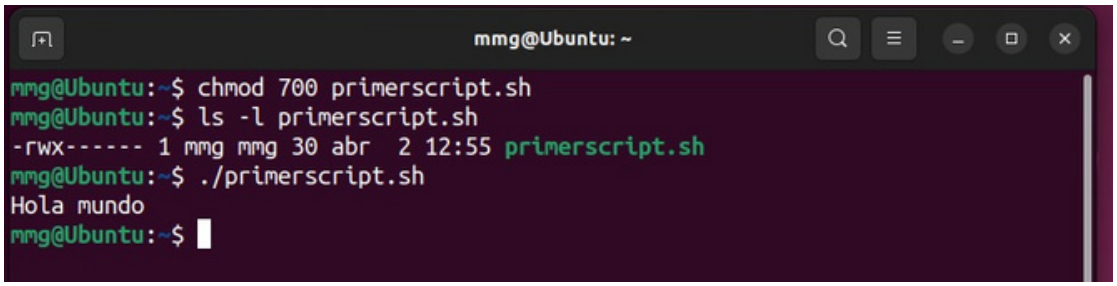


- **#!/bin/bash** → para indicar que Shell usamos, en este caso bash → SIEMPRE hay que indicarlo en la primera línea.
- Recordamos que **echo** es para escribir por pantalla.

Para poder ejecutar el fichero debemos darle permiso de ejecución, podemos hacerlo de dos formas:

- Chmod a+x primerscript.sh
- Chmod 700 primerscript.sh

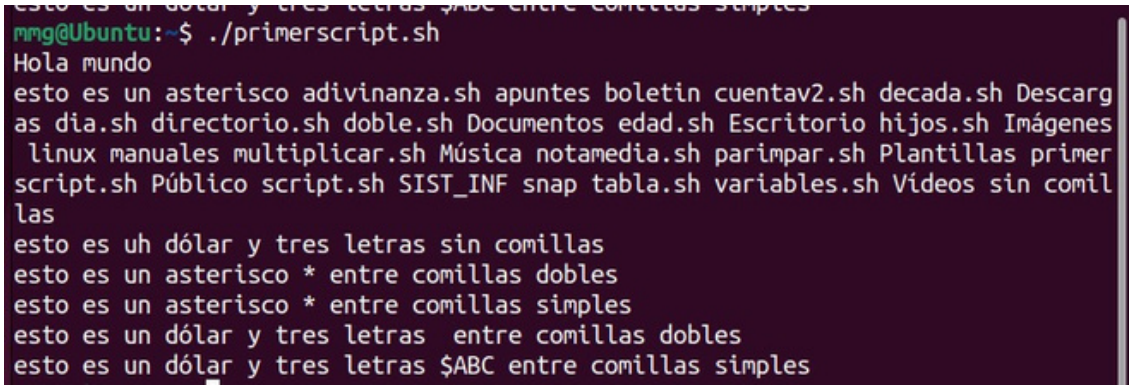
Para ejecutar el archivo tras darle los permisos → ./primerscript.sh



➢ Las comillas del "Hola mundo" realmente no son necesarias, pero es buena práctica. Si tiene caracteres especiales hay que usar comillas simples ' ' si queremos representarlos como texto.

¡Prueba lo siguiente!





➢ Si queremos ejecutar varias líneas y escribirlas en una sola → ;

➢ `Echo hola; pwd; echo Adios`

Si queremos hacer lo contrario → \

```
Echo esto\ Es una sola línea\ Aunque ocupe 3 #esto es un
comentario
```

2. Agregando variables a nuestro Script

Las variables de los Shell son muy simples:

- No tienen un tipo definido.
- No necesitan ser declaradas antes de usarlas.

Para introducir valores se usa su nombre.

Para obtener el valor: **$**nombre.

Hagamos un ejemplo.



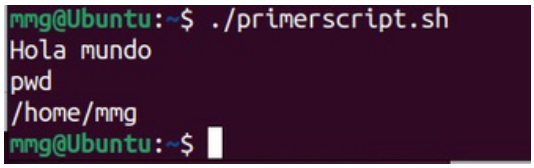
2.1. $(ORDEN)

En Ubuntu podemos usar varias expansiones en las líneas de comandos, que son muy útiles en los scripts.
Expansión = obtener el valor de la variable.

$(): permite ejecutar lo que se encuentre entre los paréntesis y devuelve su salida.

echo pwd → escribe la palabra pwd.

echo $(pwd) → ejecuta la orden pwd y escribe por pantalla su resultado.



$() = ` `

¿Qué hace el siguiente script?



1) EJERCICIO DE CLASE:

Crea un script llamado directorio.sh, donde se muestre el contenido del directorio actual. Haz uso de variables y la expansión $(). Debe ser algo similar a lo siguiente:

```
mmg@Ubuntu:~$ ./test.sh
Tu directorio contiene lo siguiente:
adivinanza.sh apuntes boletin cuentav2.sh decada.sh Descargas dia.sh direc
torio.sh doble.sh Documentos edad.sh Escritorio hijos.sh Imágenes linux ma
nuales multiplicar.sh Música notamedia.sh parimpar.sh permisos.txt Plantil
las primerscript.sh Público red020424.conf script.sh SIST_INF snap tabla.s
h test.sh variables.sh Vídeos
```

2.2. $((OPERACIÓN ARTIMÉTICA))

$(( )) = $[ ]= esta expansión se trata como una expresión aritmética, lo que está incluido entre los paréntesis, lo evalúa y devuelve su valor.

```
NUMERO=4 Echo $(($NUMERO+3))
```

NOTA: si definimos una variable como let numero=4 → nos permite realizar operaciones aritméticas, pero sin tener que usar expansiones ni dólares para las variables (cuidado que aquí no admite números decimales).

Operadores aritméticos para operaciones:

a) – para la resta.

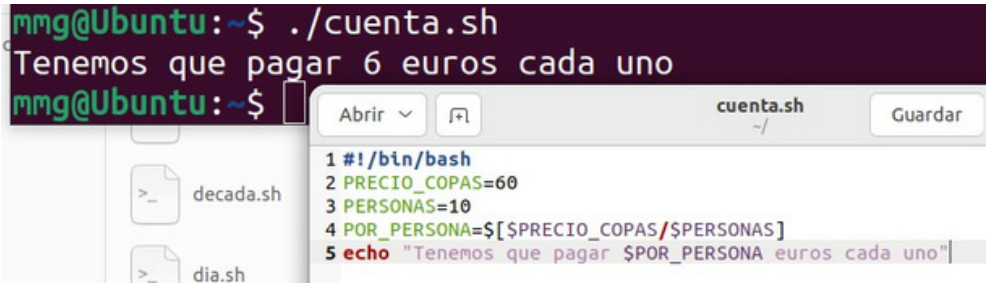
b) + para la suma.

c) / para la división.

d) * para la multiplicación.

e) % para módulo o resto.

Ejemplo cuenta.sh



2.3. ORDEN READ

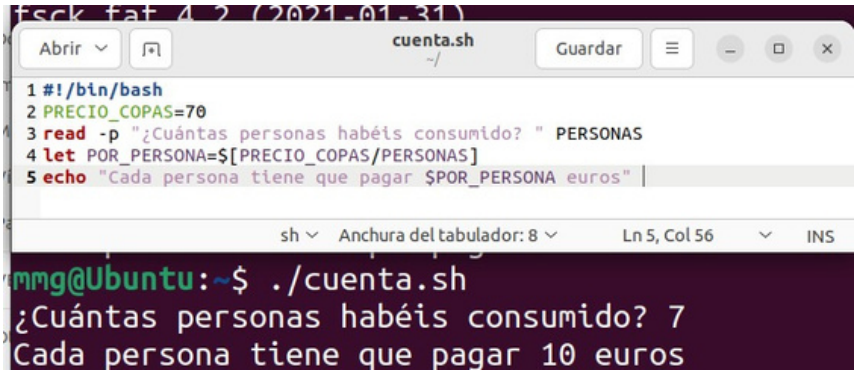
Para pedir datos al usuario.

La ejecución del script se parará y mostrará por pantalla el texto de la pregunta, y dejará que el usuario escriba la respuesta. Cuando pulse INTRO la respuesta dada se introducirá como valor de variable.

```
read -p "Texto de la pregunta" variable
```

- También puede usarse sin -p.
- Podemos hacer que lea un determinado número de caracteres, sin obligar a que el usuario pulse intro → con el parámetro -n (número de caracteres).
- Con -s → se silencia el echo (no se ve lo que el usuario escribe en la pantalla).

Ejemplo cuentav2.sh



➢ ¿Qué pasa si el resultado es un número decimal? → Bash no trabaja con decimales.

➢ Para obligar a bash a que trabaje con decimales usando el comando **bc.**

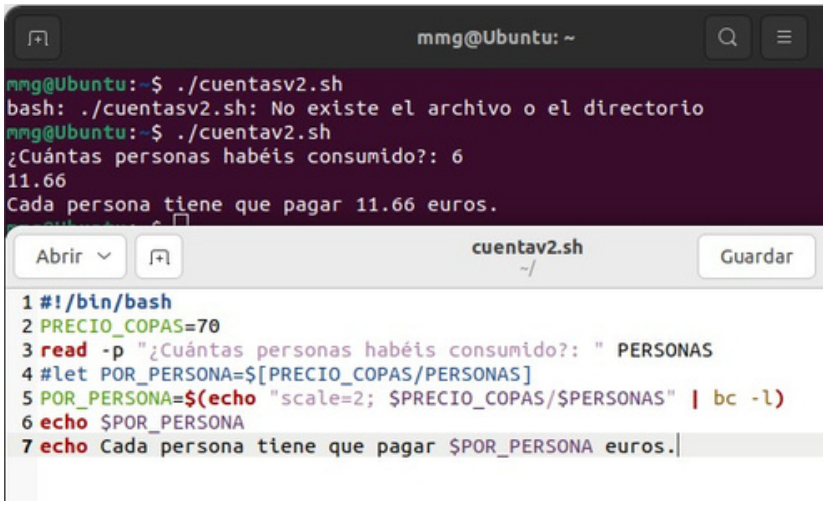
o bc: admite un gran número de parámetros aunque solo nos vamos a centrar en bc -l.

Formato:

```
$(echo "scale=2;operación" | bc -l)
```

- scale → precisión y número de decimales.
- Operación → operación aritmética.
- Bc -l → utilidad calculadora.
- -l →carga la biblioteca estándar de bc y entiende la operación.

Aplicamos el ejemplo sobre cuentav2.sh.



2) EJERCICIO DE CLASE

Crea un fichero nota media.sh:

- Pide las notas de tres prácticas de Sistemas Informáticas.
- Su nota será la media de las tres, devuélvela en la pantalla.

3. Estructuras condicionales

3.1. IF

Estructura:

```
if [ expresión ]; then
    Realizar si expresión es verdadera
Fi
```

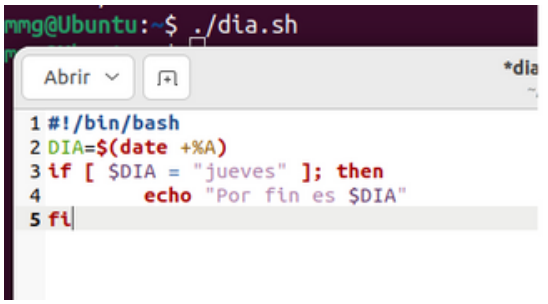
- La expresión es cualquier expresión lógica que produzca un resultado: verdadero o falso.

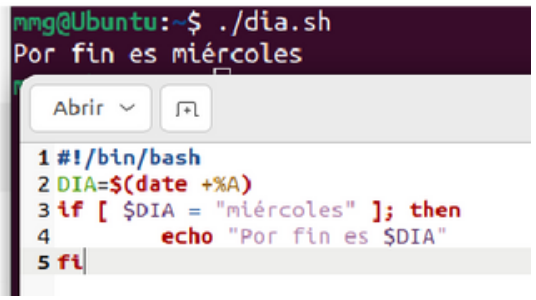
Operadores de comparación de cadenas alfanuméricas	
`Cadena1 = cadena2`	Verdadero si cadena1 es igual a cadena2
`Cadena1 != cadena2`	Verdadero si cadena1 no es igual a cadena 2.

`Cadena 1 > o < cadena 2`	Verdadero si cadena 1 es mayor o menor que cadena 2.
`-n variable1`	Verdadero si cadena1 no es nulo (tiene algún valor).
`-z variable1`	Verdadero si cadena1 es nulo (está vacía o no está definida).

```
mmg@Ubuntu:~$ if [ "maria" = "maria" ]; then echo "son iguales"; fi
son iguales
mmg@Ubuntu:~$ if [ "maria" = "mara" ]; then echo "son iguales"; fi
mmg@Ubuntu:~$
```

Ejemplo: día.sh

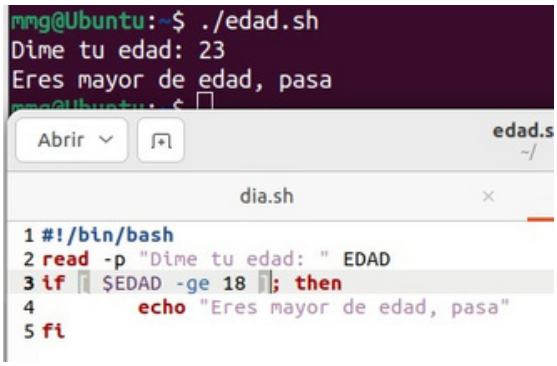




Operadores de comparación de valores numéricos	
`Numero1 -eq numero2`	Verdadero si numero1 es igual a numero2.
`Numero1 -ne numero2`	Verdadero si numero1 no es igual a numero2.
`Numero1 -lt numero2`	Verdadero si numero1 es menor a numero2.
`Numero1 -gt numero2`	Verdadero si numero1 es mayor a numero2.
`Numero1 -le numero2`	Verdadero si numero1 es menor o igual a numero2.
`Numero1 -ge numero2`	Verdadero si numero1 es mayor o igual a numero2.

Ejemplo:

Edad.sh



3) EJERCICIO DE CLASE

Crea un script donde le pida la nota al usuario y si es menor a 5 le diga "Estás suspenso".

3.2. IF..ELSE

Estructura:

```
if [ expresión1 ]; then
      Realizar si expresión1 es verdadera
else
      Realizar si expresión1 es falsa
Fi
```

Mejoramos el ejemplo anterior:

Edad.sh

```
1 #!/bin/bash
2 read -p "Dime tu edad: " EDAD
3 if [ $EDAD -ge 18 ]; then
4         echo "Eres mayor de edad, pasa"
5 else
6         echo "NO puedes pasar"
7 fi
```

Mejora el **ejercicio de clase número 3,** y si la nota es mayor o igual a 5 indica "Estás aprobado".

3.3. IF..ELIF..ELSE

Estructura:

```
if [ expresión1 ]; then
      Realizar si expresión1 es verdadera
elif [ expresión2 ]; then
      Realizar si expresión1 es falsa, ero es verdadera expresión2
elif [ expresión3 ]; then
      Realizar si expresión1 y expresión2 son falsas, pero es verdadera
      expresión3
else
      Realizar si todas las expresiones son falsas
Fi
```

Ejemplo

```
1 #!/bin/bash
2 read -p "Dime tu nora de sistemas" SISTEMAS
3 read -p "DIme tu nota de base de datos" BASE
4
5 if [ $SISTEMAS -ge 5 ] && [ $BASE -ge 5 ]; then
6         echo "Has aprobado las dos"
7 elif [ $SISTEMAS -lt 5 ] && [ $BASE -ge 5 ]; then
8         echo "Has aprobado Base de datos"
9
0         #Continua el código...
1
2 fi
```

¡Cuidado con errores comunes!

A. Cuidado con los espacios entre símbolos.

B. No usar los operadores aritméticos para cadenas.

C. No usar los operadores de cadena para números.

D. Nombre de las variables.

4) EJERCICIO DE CLASE

Crea un script que pida un número por pantalla e indique si es múltiplo de 10.

5) EJERCICIO DE CLASE

Crea un script que pida un nombre por pantalla y nos diga si empieza por letra p.

Para obtener la primera letra del nombre usaremos: cut -c 1.

6) EJERCICIO DE CLASE

Crea un script donde desarrolles un juego con una adivinanza.

7) EJERCICIO DE CLASE

Crea un script que nos pida la altura de tres centímetros y nos indica la mayor de ellas.

8) EJERCICIO DE CLASE

Crea un script que nos pida por pantalla el nombre de un mes y nos dirá si tiene 30 o 31 días ("Diciembre tiene 31 días").

4. Estructuras iterativas

Principales estructuras iterativas: for, while.

4.1. FOR

Estructura básica:

```
for variable in CONJUNTO; do
        Estas líneas se repiten cada vez por cada elemento del conjunto.
        La variable va tomando los valores del conjunto uno por uno.
done
```

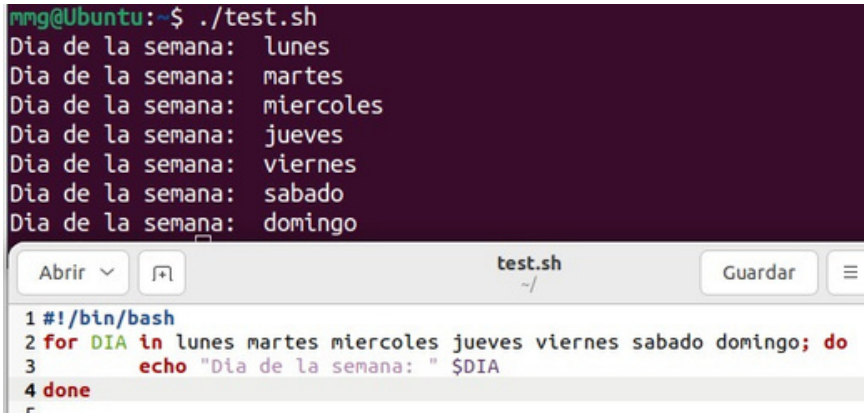
CONJUNTO: conjunto de valores cualesquiera.

- Separados por espacios en blanco.
- {x..y} → {INICIO..FINAL}

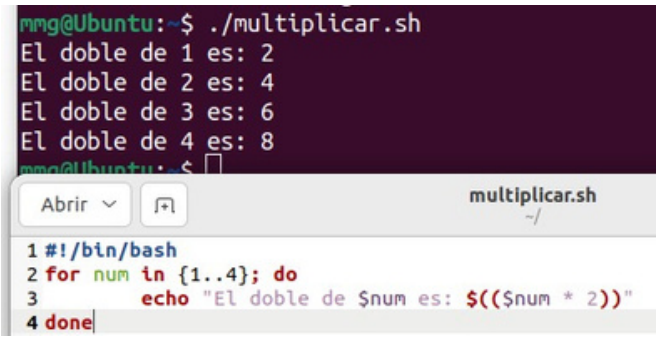
- {x..y..i} → {INICIO..FINAL..INCREMENTO}
- $(ls *.sh) → por ejemplo, ficheros del directorio con extensión sh.
- $(seq $x -1 $y).

Ejemplos:

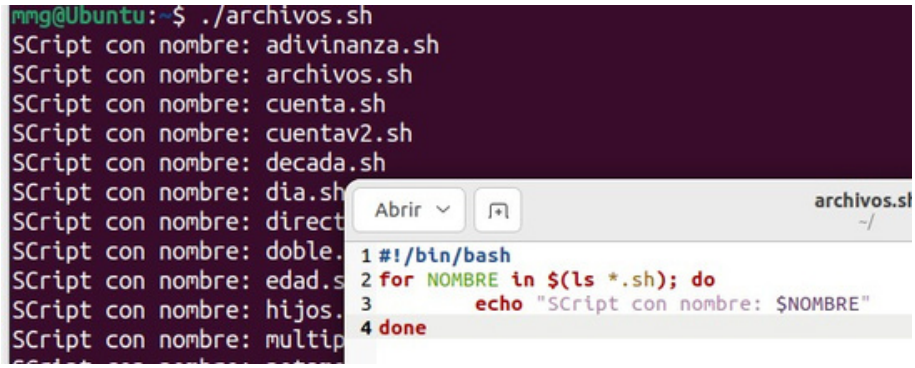
Diassemana.sh



Multiplicar.sh



Archivos.sh



9) EJERCICIO DE CLASE

Crea un archivo días.txt con los días de la semana.

Crea un script donde recorras el contenido de este y muestres por pantalla el día de la semana.

10) EJERCICIO DE CLASE

Crea un script que sume todos los números pares del 0 al 100.

11) EJERCICIO DE CLASE

Crea un script donde pida al usuario un número y devuelva su tabla de multiplicar.

4.2. WHILE

Cuando **no** queremos recorrer un conjunto de valores, si no repetir algo mientras se cumpla una condición.

Estructura básica:

```
while [ expresión ]; do
        Estas líneas se repiten MIENTRAS la expresión sea verdadera.
done
```

Ejemplo: Script que pide números y muestra el triple siempre que no se introduzca cero.



12) EJERCICIO DE CLASE

Crea un script donde se haga una pregunta al usuario, tiene tres intentos para acertar.

- Si acierta antes de que acaben los intentos, sale del bucle.
- Si se queda sin intentos sale del bucle.

Ejemplo de pregunta: ¿Quién pintó las Meninas?

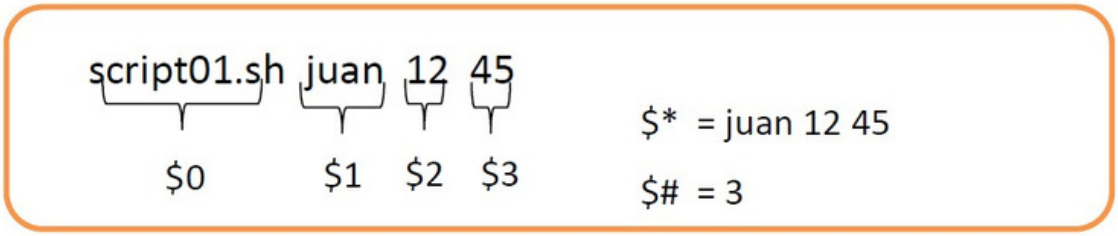
5. Paso de parámetros

- Hace referencia a la llamada de un script indicándole unos parámetros.

Los parámetros en bash se indican con un símbolo dólar ($) seguido de un número o un carácter.

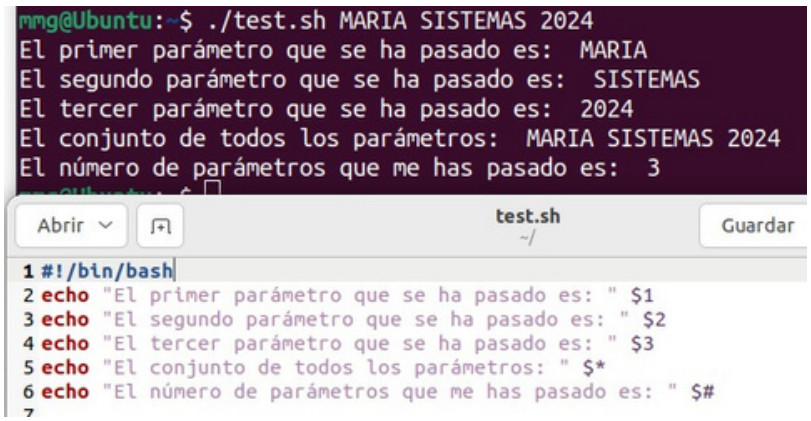
Los principales parámetros que se pueden usar son:

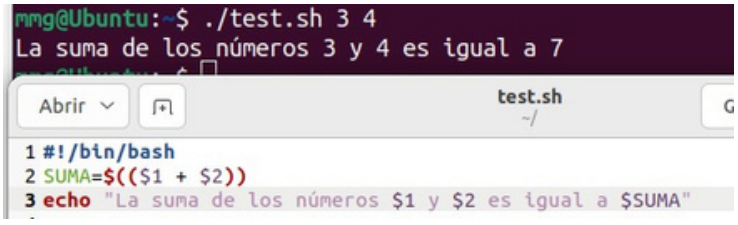
Parámetros	
$1	Devuelve el 1º parámetro pasado al script o función al ser llamado.
$2	Devuelve el 2º parámetro.
$3	Devuelve el 3º parámetro. (Podemos usar hasta $9).
$*	Devuelve todos los parámetros separados por espacio.
$#	Devuelve el número de parámetros que se han pasado.
$0	Devuelve el parámetro 0, es decir, el nombre del script o de la función.



- Además tenemos **$?** → verifica si el último comando ejecutado fue correcto, devolviendo 0 si fue correcto y 1 si no.

Ejemplos:


mmg@Ubuntu:~$ ./test.sh MARIA SISTEMAS 2024
El primer parámetro que se ha pasado es: MARIA
El segundo parámetro que se ha pasado es: SISTEMAS
El tercer parámetro que se ha pasado es: 2024
El conjunto de todos los parámetros: MARIA SISTEMAS 2024
El número de parámetros que me has pasado es: 3
Abrir
test.sh
~/
Guardar
1 #!/bin/bash
2 echo "El primer parámetro que se ha pasado es: " $1
3 echo "El segundo parámetro que se ha pasado es: " $2
4 echo "El tercer parámetro que se ha pasado es: " $3
5 echo "El conjunto de todos los parámetros: " $*
6 echo "El número de parámetros que me has pasado es: " $#


mmg@Ubuntu:~$ ./test.sh 3 4
La suma de los números 3 y 4 es igual a 7
Abrir
test.sh
~/
1 #!/bin/bash
2 SUMA=$(($1 + $2))
3 echo "La suma de los números $1 y $2 es igual a $SUMA"